

Security Audit Report

Cross-Chain NFT Marketplace using Sui, Aptos, and Solana

CHAPTER 1

Executive Summary

This report presents the security audit of the **Cross-Chain NFT Marketplace** developed using **Sui Move**, **Aptos Move**, and **Solana Anchor**. The primary objective of the audit is to evaluate the correctness, security, and reliability of the smart contracts and on-chain programs that implement NFT minting, marketplace listing, purchasing, cancellation of listings, and loyalty reward functionality across multiple blockchain platforms.

Unlike traditional centralized marketplaces, blockchain-based NFT marketplaces operate entirely through immutable smart contracts. Once deployed, contract logic cannot be modified without redeployment or governance mechanisms, making security one of the most critical aspects of decentralized application development. Any vulnerability in ownership validation, asset transfer, authorization, or state management may lead to irreversible financial losses, unauthorized asset transfers, denial of service, or permanent corruption of on-chain state. Consequently, comprehensive security assessment forms an essential component of the development lifecycle.

The audit covered the complete implementation of the marketplace on the three selected blockchain platforms. The assessment included manual source code review, execution of unit tests, deployment verification on public testnets, functional validation of marketplace operations, and platform-specific security analysis. For the Move-based implementations (Sui and Aptos), resource-oriented programming principles, object ownership, capability-based access control, and resource safety were evaluated. For the Solana implementation, the review focused on account ownership verification, Program Derived Address (PDA) validation, signer authorization, instruction constraints, rent exemption, and account serialization.

Security verification was performed through multiple stages of testing. Static analysis techniques and manual inspection were used to identify logical vulnerabilities, missing authorization checks, unsafe state transitions, and improper object handling. Functional testing confirmed the correct execution of NFT minting, marketplace listing, purchasing, and cancellation operations under both normal and invalid execution scenarios. Move unit tests were executed successfully for the Sui and Aptos implementations, while the Solana program was verified through successful deployment and transaction execution on the Solana testnet. The contracts were also evaluated for common blockchain vulnerabilities

including unauthorized asset transfers, replay attacks, double listings, integer overflow, improper ownership verification, race conditions, and denial-of-service scenarios.

The audit identified no critical or high-severity vulnerabilities that could compromise NFT ownership or marketplace integrity. Resource ownership rules enforced by the Move language significantly reduce risks associated with asset duplication and unauthorized state mutation in the Sui and Aptos implementations. Similarly, Solana's account ownership model, together with Anchor framework constraints and PDA validation, provides strong guarantees against unauthorized account modification when properly implemented.

Several medium- and low-severity observations were identified, primarily related to optimization opportunities, additional validation checks, improved event logging, and enhanced error handling. These observations do not affect the functional correctness of the marketplace but may improve maintainability, monitoring, and future scalability.

Overall, the developed marketplace demonstrates a strong security posture suitable for educational research and public testnet deployment. The implementation successfully satisfies the intended functional requirements while adhering to the security principles recommended for modern decentralized applications. Although further formal verification, third-party auditing, fuzz testing, and large-scale stress testing would be required before production deployment on a public mainnet, the current implementation provides a secure and reliable foundation for evaluating cross-chain NFT marketplace architectures.

Chapter 2: Introduction

2.1 Background

Blockchain technology has fundamentally transformed the way digital assets are created, exchanged, and managed by enabling decentralized, transparent, and tamper-resistant systems. Among the most influential innovations in this domain are **Non-Fungible Tokens (NFTs)**, which provide a standardized mechanism for representing ownership of unique digital and physical assets. Unlike cryptocurrencies that are fungible and interchangeable, NFTs possess distinct identities and metadata, making them suitable for representing digital artwork, collectibles, gaming assets, intellectual property, certificates, and tokenized real-world assets.

The rapid adoption of NFTs has led to the emergence of decentralized NFT marketplaces, where users can mint, list, buy, sell, and transfer digital assets without relying on centralized intermediaries. These marketplaces are governed by smart contracts that autonomously execute marketplace logic, including ownership verification, asset transfers, pricing mechanisms, royalty distribution, and transaction settlement. Since smart contracts operate autonomously and are generally immutable after deployment, their correctness and security directly determine the reliability of the marketplace.

As blockchain ecosystems continue to evolve, different platforms have introduced unique execution models to overcome the scalability and performance limitations of earlier blockchain networks. Among these, **Sui**, **Aptos**, and **Solana** represent three modern high-performance blockchain platforms designed to support decentralized applications with improved throughput, lower latency, and reduced transaction costs. While these platforms differ significantly in architecture, programming models, and execution strategies, each aims to provide an efficient environment for building scalable NFT marketplaces.

The project presented in this report implements a functionally equivalent NFT marketplace on all three blockchain platforms. The implementations allow users to mint NFTs, list them for sale, purchase listed assets, cancel active listings, and manage marketplace interactions while preserving ownership integrity. Since the same business logic has been implemented across multiple blockchain ecosystems, evaluating the security of each implementation is essential to ensure that platform-specific design choices do not introduce vulnerabilities or compromise the correctness of marketplace operations.

2.2 Importance of Smart Contract Security

Unlike traditional software applications, blockchain smart contracts execute within decentralized environments where transactions are irreversible and publicly verifiable. Once a contract is deployed, modifying its behavior is often difficult or impossible without

redeployment or governance mechanisms. Consequently, even minor implementation errors may lead to permanent financial losses, unauthorized asset transfers, denial of service, or corruption of on-chain state.

NFT marketplaces are particularly attractive targets for attackers because they directly manage valuable digital assets and financial transactions. A vulnerability within the marketplace logic may enable malicious users to transfer NFTs without authorization, manipulate listing prices, purchase assets without payment, create duplicate NFTs, or exploit inconsistencies in ownership tracking. Furthermore, improper validation of user permissions, inadequate state management, or incorrect handling of shared resources may compromise the integrity of the entire marketplace.

The security requirements of blockchain applications extend beyond traditional software engineering concerns. Smart contracts must ensure:

- Correct ownership verification before asset transfers.
- Proper authorization of privileged operations.
- Protection against duplicate asset creation.
- Consistent state transitions during marketplace operations.
- Prevention of unauthorized modification of marketplace data.
- Safe handling of digital assets throughout their lifecycle.
- Resistance to common blockchain attack vectors.

These requirements become even more important when comparing implementations across multiple blockchain platforms, as each ecosystem introduces different security assumptions, execution environments, and programming abstractions.

2.3 Motivation for the Security Audit

The objective of this project extends beyond implementing a functional NFT marketplace. An equally important goal is to demonstrate that the implemented marketplace satisfies fundamental security properties expected of decentralized applications operating in real blockchain environments.

Although Sui, Aptos, and Solana provide secure execution environments, application-level vulnerabilities can still arise due to incorrect implementation of marketplace logic, insufficient validation of user inputs, improper authorization checks, or inconsistent state management. Therefore, verifying the correctness of the application itself is as important as evaluating the underlying blockchain infrastructure.

The motivation for conducting this security audit is threefold.

First, the audit validates that all implemented marketplace operations—including NFT minting, listing, purchasing, and cancellation—operate correctly under expected execution scenarios without violating ownership or authorization rules.

Second, the audit evaluates platform-specific security mechanisms. The Move programming language used by Sui and Aptos introduces resource-oriented programming that enforces ownership and resource safety at the language level. In contrast, Solana employs an account-based execution model in which developers must explicitly validate account ownership, signer authorization, and Program Derived Addresses (PDAs). Understanding how these differing security models affect application robustness is an important objective of this study.

Third, the audit supports the comparative evaluation of the three blockchain platforms by identifying implementation differences, potential attack surfaces, and security trade-offs that may influence platform selection for future decentralized marketplace development.

2.4 Scope of the Audit

The scope of this security audit encompasses the complete implementation of the NFT marketplace across the three blockchain platforms. The audit focuses exclusively on the smart contract and on-chain program logic developed as part of this project and does not extend to the security of the blockchain protocols themselves.

The following functional components are included within the scope of the audit:

- NFT creation (minting)
- Marketplace listing
- NFT purchase
- Listing cancellation or delisting
- Ownership transfer
- Marketplace state management
- Loyalty and reward functionality
- User authorization and access control
- Testnet deployment and execution validation

The audit evaluates both functional correctness and security properties of these components by reviewing source code, validating transaction execution, and analysing potential attack vectors.

The assessment also includes platform-specific security mechanisms such as:

- Resource ownership verification in Move.
- Object capability validation in Sui.
- Resource account protection in Aptos.
- Account ownership verification in Solana.
- Program Derived Address (PDA) validation.
- Signer authorization.
- State transition consistency.
- Protection against unauthorized asset manipulation.

The audit does not include third-party package vulnerabilities, validator-level security, cryptographic primitive verification, consensus protocol analysis, network-layer attacks, or independent penetration testing of blockchain infrastructure, as these fall outside the scope of the developed application.

2.5 Audit Objectives

The primary objective of this audit is to determine whether the developed NFT marketplace provides a secure and reliable implementation of its intended functionality across Sui, Aptos, and Solana. To achieve this objective, the audit evaluates the application against a comprehensive set of security requirements.

The specific objectives of the audit are:

- To verify that NFTs can only be created through authorized marketplace functions.
- To ensure that only legitimate owners are permitted to list or transfer NFTs.
- To validate the correctness of ownership transfer during purchase transactions.
- To confirm that listing cancellation is restricted to the original seller.
- To verify that marketplace state remains consistent throughout all transaction flows.
- To identify vulnerabilities related to authorization, ownership validation, resource management, and state transitions.
- To evaluate the effectiveness of platform-specific security mechanisms provided by Move and Solana.
- To assess whether the implemented marketplace is suitable for deployment on public blockchain testnets.

- To identify opportunities for improving security, maintainability, and scalability in future versions of the application.

These objectives guide the subsequent security review and provide the evaluation criteria used throughout this report.

2.6 Audit Overview

The security audit was conducted using a combination of manual code inspection, functional testing, platform-specific verification tools, and live testnet validation. Each blockchain implementation was analysed independently before performing a comparative assessment of their security characteristics.

For the Sui and Aptos implementations, the audit emphasized resource safety, ownership semantics, capability-based access control, and the correctness of Move module interactions. Unit tests were executed successfully to validate expected behavior under both normal and boundary conditions.

For the Solana implementation, the review focused on the correctness of account validation, signer verification, Program Derived Address constraints, account ownership checks, and instruction execution. Testnet deployment and successful execution of marketplace operations confirmed the functional integrity of the deployed program.

The findings from these evaluations are consolidated into a comprehensive risk assessment, highlighting identified issues, implemented security controls, and recommendations for future improvements. Together, these analyses provide a holistic understanding of the security posture of the cross-chain NFT marketplace and establish a foundation for the detailed technical review presented in the subsequent chapters.

Chapter 3: System Architecture Overview

3.1 Introduction

A secure blockchain application begins with a well-designed architecture that clearly defines the interaction between users, smart contracts, and blockchain resources. Since this project implements the same NFT marketplace across three different blockchain platforms—**Sui**, **Aptos**, and **Solana**—each implementation follows the same business logic while adapting to the architectural principles and programming models of its respective blockchain.

The primary objective of the marketplace is to enable users to perform the complete lifecycle of an NFT in a decentralized environment. This includes minting new NFTs, listing them for sale, purchasing listed NFTs, cancelling active listings, and managing ownership transfers without the involvement of centralized intermediaries. While these functional requirements remain identical across all implementations, the underlying execution mechanisms differ significantly due to variations in blockchain architecture, object management, and transaction processing.

Understanding the architectural design is essential before evaluating the security of the system because many security guarantees are directly derived from how blockchain assets, accounts, resources, and permissions are managed. This chapter provides an overview of the overall marketplace architecture and discusses the platform-specific implementation details that influence the security analysis presented in later chapters.

3.2 Overall Marketplace Architecture

The developed NFT marketplace follows a modular architecture that separates asset creation, marketplace management, and transaction processing into independent functional components. Each component is responsible for a specific aspect of the marketplace while interacting with other modules through well-defined interfaces.

At a high level, the marketplace consists of four primary entities:

- Marketplace Administrator
- NFT Seller
- NFT Buyer
- Blockchain Network

The workflow begins when a creator mints a new NFT. The NFT remains under the creator's ownership until it is explicitly listed for sale. Once listed, the marketplace maintains metadata regarding the listing price, seller information, and asset identifier. Buyers may browse available listings and purchase an NFT by submitting the required payment. Upon

successful execution of the purchase transaction, ownership of the NFT is transferred to the buyer, and the seller receives the payment. Sellers also retain the ability to cancel active listings if the NFT has not yet been purchased.

Throughout this workflow, the blockchain maintains immutable records of ownership changes, transaction history, and marketplace state, ensuring transparency and preventing unauthorized modification of marketplace data.

The overall marketplace architecture can be conceptually divided into the following functional layers:

- **Application Layer:** Provides the command-line interface and client-side scripts used to invoke marketplace operations.
- **Smart Contract Layer:** Implements the NFT, marketplace, and loyalty logic using blockchain-specific programming languages.
- **Blockchain Execution Layer:** Executes transactions, validates state transitions, and maintains distributed consensus.
- **Storage Layer:** Stores NFT metadata, marketplace listings, ownership records, and transaction information on-chain.

This layered design improves modularity, simplifies testing, and allows equivalent marketplace functionality to be implemented across different blockchain ecosystems.

3.3 Sui Marketplace Architecture

The Sui implementation is built using the **Move programming language** and adopts Sui's object-centric execution model. Unlike account-based blockchains, Sui represents NFTs and marketplace entities as independent on-chain objects with unique identifiers.

The marketplace consists primarily of three Move modules:

- NFT Module
- Marketplace Module
- Loyalty Module

The **NFT module** manages the creation of NFT objects and stores metadata such as the asset name, description, image reference, and owner information. Every NFT exists as an independent object that can only have one owner at any given time.

The **Marketplace module** manages marketplace listings by storing active sale information, validating ownership before listing, processing purchase transactions, and removing listings when assets are sold or cancelled. Marketplace state is maintained using shared objects that

allow multiple users to interact with the marketplace while preserving consistency through Sui's consensus protocol.

The **Loyalty module** records user participation within the marketplace and tracks reward points generated through successful marketplace interactions. Although the loyalty system is not directly involved in NFT transfers, it demonstrates secure management of additional application-specific state.

One of the key security advantages of Sui is its object ownership model. Since every NFT is represented as an owned object, unauthorized duplication or simultaneous ownership is prevented by the underlying execution model. Furthermore, Sui distinguishes between owned objects and shared objects, allowing independent NFT transactions to execute in parallel while synchronizing operations that modify shared marketplace state.

3.4 Aptos Marketplace Architecture

The Aptos implementation also utilizes the Move programming language but follows Aptos' resource-oriented execution model. Rather than treating NFTs as independent objects, Aptos stores digital assets as Move resources associated with blockchain accounts.

The marketplace implementation is divided into multiple modules that collectively manage NFT creation, listing, purchasing, and marketplace administration.

NFTs are represented as Move resources that cannot be implicitly copied or destroyed. Resource semantics guarantee that ownership remains unique throughout the asset lifecycle, eliminating an entire class of duplication vulnerabilities commonly found in traditional software systems.

Marketplace listings are stored as on-chain resources that associate NFT identifiers with seller information and listing prices. Purchase operations validate both buyer and seller accounts before transferring ownership and updating marketplace state.

Unlike Sui, where shared objects explicitly represent marketplace state, Aptos primarily relies on account-based resource storage. The Move language enforces resource safety through compile-time verification, ensuring that marketplace resources cannot be accidentally duplicated or discarded.

Access control within the Aptos implementation is primarily enforced through signer authentication and resource ownership verification. Every transaction must be signed by the appropriate account, preventing unauthorized users from invoking privileged marketplace operations.

3.5 Solana Marketplace Architecture

The Solana implementation is developed using the **Anchor framework**, which simplifies the development of Solana smart contracts by providing declarative account validation and serialization mechanisms.

Unlike the Move-based implementations, Solana follows an account-based architecture in which all application data is stored inside blockchain accounts owned by deployed programs.

The marketplace consists of several account types, including:

- NFT metadata accounts
- Marketplace listing accounts
- Seller accounts
- Buyer accounts
- Program Derived Address (PDA) accounts

NFTs are represented through metadata associated with blockchain accounts, while marketplace listings are maintained using dedicated PDA accounts generated deterministically by the program. These PDAs prevent account collisions and ensure that marketplace state can only be modified by the deployed smart contract.

Every marketplace instruction explicitly specifies the accounts involved in transaction execution. Before processing a transaction, the program verifies:

- Account ownership
- Signer authorization
- PDA correctness
- Account mutability
- Program ownership

Unlike Move, where resource safety is enforced by the language itself, Solana requires developers to manually validate account relationships and permissions. Consequently, security depends heavily on the correctness of account validation logic implemented within each instruction.

3.6 Comparative Architectural Analysis

Although the business functionality remains identical across all three implementations, the architectural differences significantly influence both application security and execution behavior.

Sui adopts an object-oriented execution model in which every NFT exists as an independently owned object. This design enables fine-grained ownership tracking and supports parallel execution for transactions involving unrelated objects. Shared marketplace state is managed through shared objects that synchronize concurrent access while preserving consistency.

Aptos employs a resource-oriented architecture based on Move's linear type system. Resources cannot be copied or implicitly discarded, ensuring that digital assets always have a single owner. Marketplace state is managed through account-associated resources, with signer authentication serving as the primary authorization mechanism.

Solana follows a fundamentally different account-based model. Rather than relying on language-level ownership guarantees, Solana stores application state inside explicitly managed accounts. Developers must manually enforce ownership validation, signer verification, and account constraints, providing greater flexibility but increasing implementation complexity.

These architectural differences also influence the types of vulnerabilities that may arise. Move-based systems naturally eliminate several classes of ownership-related bugs through compile-time resource safety, whereas Solana places greater responsibility on developers to correctly implement account validation and permission checks.

Despite these differences, all three implementations successfully support the core marketplace workflow while maintaining secure ownership transfer and consistent marketplace state. The following chapters examine how these architectural choices affect the security properties of each implementation through detailed audit and threat analysis.

Chapter 4: Audit Methodology

4.1 Introduction

A comprehensive security audit requires a systematic and repeatable evaluation process that examines not only the correctness of smart contract code but also its behavior during execution under realistic operating conditions. Since blockchain applications manage immutable digital assets and irreversible financial transactions, traditional software testing alone is insufficient to establish confidence in their security. Instead, multiple complementary verification techniques must be employed to identify implementation errors, authorization weaknesses, incorrect state transitions, and platform-specific vulnerabilities.

The security audit presented in this report adopts a multi-layered methodology that combines manual code review, static analysis, functional testing, platform-specific verification tools, and live testnet deployment. Rather than relying on a single verification technique, the audit integrates several independent assessment methods to evaluate the marketplace from both a security and reliability perspective.

Because the project implements the same NFT marketplace across Sui, Aptos, and Solana, the methodology was designed to ensure consistency across all three platforms while accounting for their architectural differences. This enables a fair comparison of the security mechanisms provided by each blockchain ecosystem and helps identify implementation-specific risks that may arise from differences in programming models or execution environments.

4.2 Audit Objectives

The methodology was developed to achieve the following primary objectives:

- Verify the functional correctness of all marketplace operations.
- Ensure that ownership of NFTs is preserved throughout their lifecycle.
- Confirm that only authorized users can execute privileged marketplace operations.
- Validate the correctness of state transitions during listing, purchasing, and cancellation.
- Identify implementation flaws that could compromise marketplace integrity.
- Evaluate blockchain-specific security mechanisms provided by Sui, Aptos, and Solana.
- Assess the robustness of the deployed contracts under realistic testnet execution.
- Document observations, limitations, and recommendations for future improvements.

These objectives guided every stage of the audit process and served as the basis for evaluating the overall security posture of the developed marketplace.

4.3 Audit Scope

The audit covers all smart contract modules and on-chain programs developed as part of this project. The assessment focuses on the application logic responsible for implementing marketplace functionality rather than the underlying blockchain protocols themselves.

The following components were included within the scope of the audit:

Functional Components

- NFT creation (minting)
- Marketplace listing
- NFT purchasing
- Listing cancellation (delisting)
- Ownership transfer
- Loyalty reward management
- Marketplace state management

Security Components

- Access control
- Authorization mechanisms
- Ownership verification
- Input validation
- State consistency
- Resource management
- Account validation
- Transaction execution

Deployment Components

- Smart contract compilation
- Testnet deployment
- Transaction execution

- CLI interaction
- Gas consumption analysis
- Performance observation

The audit does not evaluate consensus algorithms, validator security, peer-to-peer networking, cryptographic primitives, or vulnerabilities within the blockchain protocols themselves, as these are managed by the respective blockchain platforms.

4.4 Security Assessment Approach

The audit follows a layered assessment strategy in which each implementation undergoes multiple forms of verification before being considered secure.

The assessment process consists of five major stages:

1. Source Code Review
2. Static Verification
3. Functional Testing
4. Testnet Validation
5. Comparative Security Analysis

Each stage evaluates different aspects of the marketplace and collectively provides a comprehensive understanding of the application's security characteristics.

4.5 Manual Source Code Review

Manual inspection formed the foundation of the security audit. Every smart contract module and program instruction was examined to verify the correctness of business logic and identify vulnerabilities that automated tools may overlook.

The review focused on several critical security properties.

Access Control

The audit verified that privileged functions can only be executed by authorized users.

Examples include:

- Only NFT owners may list assets.
- Only sellers may cancel listings.
- Buyers cannot modify marketplace records.

- Unauthorized users cannot mint NFTs on behalf of others.

Ownership Validation

Ownership verification was examined throughout every marketplace operation.

The review confirmed that:

- NFTs cannot be transferred without owner authorization.
- Ownership changes occur only after successful purchases.
- Ownership remains unique throughout the NFT lifecycle.
- Duplicate ownership cannot occur.

State Transition Analysis

Every marketplace operation changes blockchain state.

The audit verified that:

Mint

↓

Owned NFT

↓

Listed NFT

↓

Purchased NFT

↓

Transferred Ownership

or

Listed NFT

↓

Cancelled Listing

↓

Returned to Owner

The review confirmed that no invalid intermediate states could occur during transaction execution.

Business Logic Verification

Marketplace rules were manually verified to ensure consistency.

Examples include:

- NFTs cannot be purchased twice.
- Cancelled listings disappear from the marketplace.
- Purchase updates ownership records.
- Listing prices remain unchanged unless explicitly modified.
- Marketplace data remains synchronized.

4.6 Static Verification

Static verification analyses source code without executing transactions.

Its objective is to detect logical inconsistencies, improper resource handling, unused variables, unreachable code, and potential implementation mistakes.

Different verification techniques were used for each blockchain due to differences in programming languages and development frameworks.

Sui

For the Sui implementation, static verification included:

- Move compiler validation
- Module dependency verification
- Type checking
- Resource safety verification
- Ownership validation
- Object consistency analysis

The successful execution of

`sui move build`

confirmed that:

- Modules compile successfully.
- Resource ownership rules are satisfied.
- Type safety is preserved.
- Module interfaces are correctly defined.

Aptos

For Aptos, verification included:

- Move compiler
- Resource checking
- Signer validation
- Module dependency analysis
- Type verification

Compilation confirmed that:

- Resources are properly managed.
- Ownership constraints are satisfied.
- Invalid resource copies are prevented.
- Module interfaces are consistent.

Solana

The Solana implementation underwent static verification using the Anchor framework.

Verification included:

- Program compilation
- Account constraint checking
- PDA derivation validation
- Serialization verification
- Instruction validation

Compilation verified that:

- Account definitions are consistent.
- Program instructions compile successfully.
- PDA derivation is deterministic.
- Instruction constraints satisfy Anchor requirements.

4.7 Functional Testing

Functional testing was conducted to verify that every marketplace feature behaves correctly under normal operating conditions.

Each blockchain implementation underwent the complete NFT lifecycle.

The following operations were executed successfully.

NFT Minting

Objective:

Verify successful creation of NFTs.

Validation:

- NFT created successfully.
- Owner recorded correctly.
- Metadata stored.
- Unique identifier generated.

Marketplace Listing

Objective:

Verify successful listing of NFTs.

Validation:

- Listing created.
- Seller verified.
- Marketplace updated.
- NFT becomes available for purchase.

NFT Purchase

Objective:

Verify secure ownership transfer.

Validation:

- Buyer pays required amount.
- Ownership transferred.
- Listing removed.
- Marketplace updated.

Listing Cancellation

Objective:

Verify seller can remove listings.

Validation:

- Only seller can cancel.
- Listing removed.
- NFT returned to owner.
- Marketplace state updated.

Negative test scenarios were also evaluated where applicable.

Examples included:

- Invalid owner attempts listing.
- Non-owner attempts cancellation.
- Purchasing unavailable NFTs.
- Invalid transaction parameters.

These scenarios confirmed that unauthorized operations were rejected by the smart contracts.

4.8 Platform-Specific Verification

Since each blockchain provides different security guarantees, additional verification procedures were performed.

Sui Verification

The following commands were successfully executed.

sui move build

sui move test

Testnet deployment verified:

- NFT minting
- Listing
- Buying
- Cancellation

The audit also confirmed correct behavior of:

- Object ownership
- Shared objects
- Dynamic fields
- Capability checks

Aptos Verification

Verification included:

- Module compilation
- Move unit testing
- Resource ownership validation
- Signer verification
- Testnet transaction execution

Marketplace operations completed successfully without ownership inconsistencies.

Solana Verification

Verification included:

- Anchor program deployment
- Program initialization

- PDA validation
- Signer verification
- Transaction execution
- Account ownership verification

Marketplace operations executed successfully on the Solana testnet.

4.9 Testnet Deployment Validation

Security cannot be fully evaluated through local testing alone.

Therefore, every implementation was deployed on its respective public testnet.

Deployment validation confirms that:

- Smart contracts compile successfully.
- Programs deploy without errors.
- Transactions execute correctly.
- Blockchain state remains consistent.
- Marketplace functionality operates under real network conditions.

Successful testnet deployment provides additional confidence that the implementations behave correctly in distributed environments involving network latency, validator consensus, and persistent on-chain storage.

4.10 Limitations of the Audit Methodology

Although the adopted methodology provides comprehensive coverage of the developed application, certain limitations should be acknowledged.

The audit primarily evaluates application-level security rather than the underlying blockchain infrastructure. Consequently, consensus protocols, validator behavior, networking, and cryptographic algorithms are assumed to operate correctly as provided by the respective blockchain platforms.

Furthermore, the assessment was conducted using public testnets, where network conditions, validator performance, and transaction confirmation times may differ from production environments. The audit also focuses on sequential transaction execution and does not include large-scale concurrent stress testing or adversarial network simulations.

Finally, while manual review and functional testing provide strong assurance of correctness, this audit does not constitute a formal third-party certification. Additional techniques such as formal verification, fuzz testing, symbolic execution, and independent external audits would further strengthen confidence before deploying the marketplace to a production mainnet.

Chapter 5: Threat Model

5.1 Introduction

A threat model provides a structured approach for identifying potential security risks that may affect a blockchain application. Rather than focusing solely on existing vulnerabilities, a threat model anticipates possible attack scenarios by examining the assets being protected, the actors capable of performing malicious actions, the available attack surfaces, and the security controls implemented to mitigate these threats.

For decentralized applications such as NFT marketplaces, the threat landscape differs significantly from that of conventional web applications. Since transactions are immutable and executed autonomously by smart contracts, any flaw in the application logic may have irreversible consequences. Unlike centralized systems where administrators can reverse fraudulent transactions or restore compromised data, blockchain systems permanently record every successful transaction. Therefore, preventing vulnerabilities before deployment is considerably more important than detecting them after exploitation.

The objective of this chapter is to identify the critical assets managed by the marketplace, define the assumptions under which the application operates, classify potential threat actors, analyse common attack vectors relevant to the developed marketplace, and evaluate the effectiveness of the implemented security mechanisms in mitigating these risks.

5.2 Security Objectives

The security architecture of the NFT marketplace was designed to achieve several fundamental objectives that ensure the integrity, confidentiality (where applicable), and availability of marketplace operations.

The primary security objectives are as follows:

- Preserve the uniqueness and ownership of every NFT.
- Prevent unauthorized minting or transfer of digital assets.
- Ensure that only legitimate owners can list or delist NFTs.
- Guarantee that ownership changes occur only after successful purchase transactions.
- Protect marketplace state from unauthorized modification.
- Prevent duplicate listings and inconsistent marketplace records.
- Maintain transaction integrity throughout the NFT lifecycle.
- Ensure that invalid or malicious transactions are rejected by the smart contracts.

- Preserve consistency of on-chain data under concurrent user interactions.
- Minimize opportunities for financial exploitation arising from logical vulnerabilities.

These objectives form the basis against which potential threats are evaluated throughout this audit.

5.3 Protected Assets

The marketplace manages several valuable on-chain assets that require protection against unauthorized access or manipulation. These assets include both digital resources and application state maintained by the smart contracts.

The primary protected assets are summarized in Table 5.1.

Table 5.1 Protected Assets

Asset	Description	Security Requirement
NFT Objects	Unique digital assets created by users	Ownership integrity
Marketplace Listings	Active sale records	Consistency and authenticity
User Accounts	Blockchain identities of buyers and sellers	Authentication
Marketplace State	Shared marketplace information	Integrity
Loyalty Data	Reward points associated with users	Accuracy
Transaction Records	On-chain transaction history	Immutability
Smart Contract Code	Marketplace business logic	Correctness
Digital Payments	Funds exchanged during purchases	Secure transfer

Each of these assets represents a potential target for malicious actors. Consequently, the marketplace incorporates multiple security mechanisms to ensure that these assets remain protected throughout transaction execution.

5.4 Trust Assumptions

The security analysis assumes that the underlying blockchain platforms operate according to their documented specifications. The following assumptions define the boundaries of the security audit.

Blockchain Security

It is assumed that:

- Validator consensus operates correctly.
- Cryptographic algorithms remain secure.
- Transaction signatures cannot be forged.
- Blockchain history cannot be modified after confirmation.

These assumptions are inherited from the security guarantees provided by Sui, Aptos, and Solana.

User Authentication

The marketplace assumes that users securely manage their private keys.

Compromise of a user's wallet or private key lies outside the scope of the marketplace implementation.

If a user's wallet is compromised, an attacker may legitimately authorize transactions using the stolen credentials. This represents a wallet security issue rather than a smart contract vulnerability.

Smart Contract Integrity

The audit assumes that deployed smart contracts are identical to the reviewed source code.

No unauthorized upgrades or modifications occur after deployment.

Network Availability

The marketplace assumes normal operation of the public blockchain network.

Temporary network congestion or validator delays may affect transaction latency but do not compromise marketplace correctness.

5.5 Threat Actors

Several categories of adversaries were considered during the security assessment.

External Attacker

An external attacker has no legitimate ownership of marketplace assets but attempts to exploit vulnerabilities in smart contract logic.

Possible objectives include:

- Stealing NFTs
- Creating duplicate assets
- Manipulating marketplace records
- Exploiting authorization flaws

Malicious Seller

A legitimate marketplace participant may attempt to abuse marketplace functionality.

Potential attacks include:

- Selling NFTs not owned by the seller.
- Creating duplicate listings.
- Cancelling completed sales.
- Manipulating listing information.

Malicious Buyer

A buyer may attempt to obtain NFTs without satisfying marketplace requirements.

Possible attacks include:

- Purchasing assets without payment.
- Buying cancelled listings.
- Purchasing the same NFT multiple times.
- Manipulating transaction parameters.

Curious User

A technically knowledgeable user may intentionally invoke marketplace functions using unexpected parameters.

Examples include:

- Invalid object identifiers.
- Incorrect account references.
- Modified transaction inputs.
- Boundary values.

Such users help identify implementation weaknesses that ordinary users would never encounter.

Automated Bots

Although the marketplace currently operates on public testnets, automated scripts may rapidly submit transactions attempting to:

- Spam marketplace operations.
- Discover logical inconsistencies.
- Exploit race conditions.
- Generate denial-of-service conditions.

5.6 Attack Surface Analysis

The attack surface represents every interface through which a malicious actor may interact with the marketplace.

The primary attack surfaces include:

Smart Contract Entry Functions

Public functions including:

- `mint_nft()`
- `list_nft()`
- `buy_nft()`
- `cancel_listing()`

Every public function must validate:

- Caller authorization
- Input parameters
- Ownership
- State consistency

Transaction Inputs

Attackers may manipulate:

- NFT identifiers
- Prices
- Addresses
- Object references
- Account parameters

Proper validation prevents malformed transactions from altering marketplace state.

Marketplace State

Marketplace listings represent shared application state.

Potential attacks include:

- Duplicate listings
- Invalid listing removal
- Unauthorized updates
- Inconsistent marketplace records

Ownership Transfer

Ownership transfer represents one of the highest-risk operations because it changes the legal owner of a digital asset.

The audit verifies that transfers occur only:

- After successful payment.

- Between authorized participants.
- Exactly once.

Deployment Configuration

Incorrect deployment may expose risks including:

- Incorrect package identifiers.
- Invalid program ownership.
- Misconfigured capabilities.
- Incorrect PDA derivation.

Deployment verification therefore forms an important part of the security assessment.

5.7 Threat Analysis

The marketplace was evaluated against several well-known blockchain attack vectors.

Unauthorized Minting

Description

An attacker attempts to create NFTs without following the intended minting process.

Potential Impact

- Duplicate assets
- Marketplace corruption
- Loss of trust

Mitigation

- Controlled mint functions.
- Resource creation validation.
- Ownership assignment during minting.

Risk Level: Low

Unauthorized Listing

Description

An attacker attempts to list another user's NFT.

Potential Impact

- Asset theft
- Fraudulent sales

Mitigation

- Ownership verification.
- Signer validation.
- Object ownership checks.

Risk Level: Low

Unauthorized Purchase

Description

An attacker attempts to obtain an NFT without satisfying purchase requirements.

Potential Impact

- Financial loss
- Ownership corruption

Mitigation

- Transaction validation.
- Ownership transfer only after successful execution.
- Marketplace consistency checks.

Risk Level: Low

Double Listing

Description

An NFT is listed multiple times simultaneously.

Potential Impact

- Marketplace inconsistency.

- Multiple buyers.
- Duplicate sales.

Mitigation

- Active listing validation.
- Marketplace state verification.
- Listing removal after purchase.

Risk Level: Low

Replay Attack**Description**

An attacker attempts to replay previously executed transactions.

Potential Impact

- Duplicate purchases.
- Duplicate transfers.

Mitigation

Replay protection is provided by blockchain transaction sequencing and nonce mechanisms implemented by the underlying blockchain platforms.

Risk Level: Very Low

Integer Overflow**Description**

Arithmetic operations exceed supported numeric ranges.

Potential Impact

- Price manipulation.
- Reward calculation errors.

Mitigation

Modern Move and Rust implementations provide strong type safety and checked arithmetic where appropriate, substantially reducing overflow risks.

Risk Level: Very Low

Unauthorized Cancellation

Description

A user attempts to cancel another seller's listing.

Potential Impact

- Marketplace disruption.

Mitigation

- Seller verification.
- Ownership validation.
- Authorization checks.

Risk Level: Low

Denial-of-Service (DoS)

Description

Attackers submit excessive transactions attempting to exhaust marketplace resources.

Potential Impact

- Delayed transaction execution.
- Increased confirmation times.

Mitigation

- Blockchain gas fees.
- Validator transaction limits.
- Consensus scheduling.

Risk Level: Medium

State Corruption

Description

Unexpected transaction ordering produces inconsistent marketplace data.

Potential Impact

- Lost NFTs.
- Invalid listings.
- Ownership inconsistencies.

Mitigation

- Atomic transaction execution.
- Blockchain consensus.
- Resource ownership guarantees.
- Account validation.

Risk Level: Low

5.8 Risk Assessment Matrix

The identified threats were evaluated based on their likelihood of occurrence and their potential impact on the marketplace.

Table 5.2 Risk Assessment Matrix

Threat	Likelihood	Impact	Risk
Unauthorized Minting	Low	High	Low
Unauthorized Listing	Low	High	Low
Unauthorized Purchase	Low	High	Low
Duplicate Listing	Low	Medium	Low
Replay Attack	Very Low	Medium	Very Low
Integer Overflow	Very Low	Medium	Very Low
Unauthorized Cancellation	Low	Medium	Low
State Corruption	Low	High	Low
Denial-of-Service	Medium	Medium	Medium
Invalid Transaction Input	Medium	Low	Low

The assessment indicates that the implemented marketplace is resilient against the majority of common application-level threats. The only medium-risk category identified is denial-of-

service, which is primarily mitigated by the blockchain infrastructure through transaction fees, validator scheduling, and network-level protections rather than by application logic alone.

5.9 Summary

The threat model establishes the security context for the remainder of this audit by identifying the critical assets protected by the NFT marketplace, the assumptions under which the application operates, the potential threat actors, and the primary attack vectors relevant to decentralized marketplace systems. The analysis demonstrates that the implemented security controls—including ownership verification, signer authentication, resource safety, account validation, and blockchain-level consensus—effectively mitigate the most significant risks associated with NFT marketplace operations.

The subsequent chapters build upon this threat model by performing detailed security reviews of each blockchain implementation. Each review evaluates how the platform-specific architecture and programming model contribute to mitigating the identified threats and preserving the integrity of marketplace operations.

Chapter 6: Sui Smart Contract Security Review

6.1 Introduction

The Sui implementation of the NFT marketplace was developed using the **Move programming language** and deployed on the **Sui Testnet**. Unlike traditional account-based blockchains, Sui employs an object-centric execution model in which every NFT and marketplace component exists as an independent on-chain object with a unique identifier. This architectural approach enables efficient ownership tracking, fine-grained access control, and parallel transaction execution while reducing the likelihood of common asset management vulnerabilities.

The objective of this chapter is to evaluate the security of the Sui implementation by analysing the smart contract modules, ownership model, authorization mechanisms, object management, transaction execution, and deployment behaviour. The review assesses whether the implementation satisfies the security objectives defined in the previous chapters and identifies any potential weaknesses that could affect marketplace integrity.

The audit focuses on three primary Move modules developed for this project:

- **NFT Module**
- **Marketplace Module**
- **Loyalty Module**

Each module is reviewed independently before analysing the security of their interactions during complete marketplace operations.

6.2 Sui Security Architecture

The Sui blockchain adopts an object-oriented programming model in which every on-chain asset is represented as an object rather than being stored directly inside user accounts. Every object possesses a globally unique identifier and ownership metadata maintained by the blockchain.

The marketplace architecture consists of the following object types:

Object	Purpose
NFT Object	Represents an individual NFT
Marketplace Object	Maintains marketplace listings
Listing Object	Stores NFT sale information

Object	Purpose
Loyalty Object	Maintains reward points
Coin Object	Represents payment tokens

This object-based design provides several security advantages:

- Each NFT has a unique owner.
- Objects cannot exist simultaneously under multiple owners.
- Object ownership is validated automatically by the Move runtime.
- Invalid ownership transfers are rejected before execution.
- Independent objects can execute transactions concurrently.

These properties significantly reduce the likelihood of ownership-related vulnerabilities.

6.3 NFT Module Security Review

The NFT module is responsible for creating new NFT objects and assigning ownership to the creator.

Security Objectives

The module must guarantee that:

- Every NFT receives a unique identifier.
- NFTs cannot be duplicated.
- Ownership is assigned correctly during minting.
- Metadata remains associated with the NFT.
- Unauthorized users cannot modify existing NFTs.

Audit Findings

The minting function successfully creates a new NFT object during each invocation.

The audit verified that:

- NFT identifiers are unique.
- Ownership is transferred directly to the transaction sender.
- Metadata fields remain immutable after creation unless explicitly designed otherwise.

- Object creation follows Move's resource semantics.

Since NFTs are represented as Move objects with linear ownership, duplication through accidental copying is impossible.

Security Assessment

Property	Result
Duplicate NFT Prevention	Passed
Ownership Assignment	Passed
Metadata Integrity	Passed
Unauthorized Modification	Passed

Risk Level: Very Low

6.4 Marketplace Module Security Review

The Marketplace module manages NFT listings, purchases, and cancellations.

Because this module handles ownership transitions and marketplace state, it represents the highest-risk component of the application.

The audit evaluated every public entry function.

6.4.1 Listing Operation

The listing operation transfers marketplace information into a shared marketplace object while maintaining NFT ownership until purchase.

The following checks were verified:

- Seller owns the NFT.
- NFT is valid.
- Listing price is valid.
- Listing is recorded correctly.
- Marketplace state updates successfully.

Potential Risks

Possible vulnerabilities include:

- Listing another user's NFT.
- Duplicate marketplace entries.
- Invalid price manipulation.

Audit Result

Ownership validation prevents unauthorized listing.

Marketplace records remain consistent.

No duplicate listing behaviour was observed during testing.

Status: Passed

6.4.2 Purchase Operation

The purchase function represents the most critical transaction in the marketplace.

The function performs:

- Buyer verification.
- Payment validation.
- NFT transfer.
- Marketplace update.
- Listing removal.

The audit confirmed that ownership changes only after successful transaction execution.

If transaction execution fails at any stage, blockchain atomicity guarantees rollback of all intermediate state changes.

Security Properties

Property

Ownership Validation

Payment Verification

Atomic Transfer

Listing Removal

State Consistency

Risk Level: Low

6.4.3 Listing Cancellation

Listing cancellation allows sellers to remove active marketplace listings.

The audit confirmed:

- Only original sellers can cancel listings.
- Marketplace records are removed correctly.
- NFT ownership remains unchanged.
- Invalid cancellation attempts are rejected.

No unauthorized cancellation behaviour was observed.

6.5 Loyalty Module Security Review

The Loyalty module maintains reward points associated with marketplace activity.

Although it does not directly participate in NFT ownership transfer, incorrect implementation could compromise marketplace incentives.

The audit verified:

- Reward updates occur after valid marketplace operations.
- Users cannot arbitrarily modify reward balances.
- Reward calculations remain deterministic.
- Marketplace state remains independent of loyalty state.

The module exhibited no observable security concerns during testing.

6.6 Access Control Review

Authorization represents one of the most important aspects of smart contract security.

The audit examined every public entry function to verify that privileged operations require appropriate authorization.

The following access-control mechanisms were evaluated:

Operation	Authorization Required
-----------	------------------------

Mint NFT	Transaction signer
----------	--------------------

List NFT	NFT owner
----------	-----------

Buy NFT	Buyer signature
---------	-----------------

Cancel Listing	Seller signature
----------------	------------------

Loyalty Update	Internal contract logic
----------------	-------------------------

No privilege escalation vulnerabilities were identified.

6.7 Object Ownership Verification

The primary security advantage of Sui lies in its object ownership model.

Every NFT exists as an owned object that can belong to only one account at any given time.

The audit confirmed:

- Objects cannot be duplicated.
- Ownership cannot exist simultaneously under multiple users.
- Invalid object references are rejected.
- Ownership transfers occur exactly once.

The Move runtime enforces these guarantees automatically during transaction execution.

This significantly reduces implementation complexity compared to traditional account-based systems.

6.8 Shared Object Security

Marketplace listings are maintained using shared objects.

Shared objects introduce additional complexity because multiple users may attempt to modify marketplace state concurrently.

The audit verified:

- Shared objects remain synchronized.
- Listing updates occur atomically.

- Invalid concurrent modifications are rejected.
- Marketplace consistency is preserved.

The observed latency during listing and cancellation reflects consensus coordination required for shared object updates but does not indicate a security weakness.

6.9 Resource Safety Analysis

Move's resource-oriented programming model provides compile-time guarantees that eliminate several common smart contract vulnerabilities.

The audit confirmed that:

- Resources cannot be copied.
- Resources cannot be accidentally discarded.
- Ownership remains unique.
- Assets cannot disappear unexpectedly.

These guarantees substantially reduce risks including:

- Double spending.
- Duplicate NFTs.
- Lost ownership.
- Asset leakage.

This represents one of the strongest security characteristics of the Sui implementation.

6.10 Testnet Security Validation

Following successful compilation and unit testing, the marketplace was deployed on the Sui Testnet for real-world validation.

The following commands completed successfully:

- `sui move build`
- `sui move test`

Subsequent live transaction execution verified:

- NFT Minting
- Marketplace Listing

- NFT Purchase
- Listing Cancellation

Every transaction completed successfully without unexpected state transitions or ownership inconsistencies.

Observed transaction effects confirmed that object ownership, marketplace updates, and storage allocation behaved as intended throughout the complete NFT lifecycle.

6.11 Security Observations

During the audit, several positive implementation characteristics were identified:

Strengths

- Strong object ownership model.
- Automatic resource safety.
- Compile-time ownership verification.
- Capability-based authorization.
- Atomic transaction execution.
- Deterministic object identifiers.
- Prevention of duplicate ownership.
- Secure shared object synchronization.
- Successful testnet validation.

Minor Observations

No critical security vulnerabilities were identified.

However, several improvements could further strengthen the implementation:

- Additional event emission for marketplace analytics.
- More descriptive custom error messages.
- Expanded unit test coverage for invalid transaction scenarios.
- Property-based testing for boundary conditions.
- Formal verification of business logic using advanced Move Prover specifications where applicable.

These observations relate primarily to maintainability and assurance rather than functional correctness.

6.12 Sui Security Summary

The security review demonstrates that the Sui implementation provides a robust and reliable foundation for the NFT marketplace. The combination of Move's resource-oriented programming model and Sui's object-centric execution architecture offers strong guarantees regarding ownership integrity, authorization enforcement, and transaction consistency. Throughout compilation, unit testing, and public testnet deployment, the implementation successfully preserved NFT ownership, prevented unauthorized marketplace operations, and maintained consistent marketplace state.

No critical or high-severity vulnerabilities were identified during the audit. The implemented controls effectively mitigate common blockchain risks such as unauthorized asset transfer, duplicate ownership, inconsistent state transitions, and invalid object manipulation. The few observations noted are limited to potential enhancements in testing, event logging, and developer tooling rather than security flaws.

Overall, the Sui implementation exhibits a **high level of application security** and is well suited for educational research, performance benchmarking, and public testnet deployment. Its architecture also provides a strong basis for future enhancements and potential production hardening through formal verification, extended testing, and independent third-party audits.

Chapter 7: Aptos Smart Contract Security Review

7.1 Introduction

The Aptos implementation of the NFT marketplace was developed using the **Move programming language** and deployed on the **Aptos Testnet**. Although Aptos and Sui share the same programming language foundation, their execution models differ in several important aspects. Aptos follows a **resource-oriented account model**, where digital assets and marketplace data are stored as Move resources under blockchain accounts rather than as independently owned objects.

The primary objective of this chapter is to evaluate the security of the Aptos implementation by examining its smart contract modules, resource management mechanisms, authorization model, signer validation, transaction execution, and state consistency. The audit also evaluates whether the implementation leverages the security guarantees provided by the Move language while ensuring that application-specific marketplace logic does not introduce vulnerabilities.

The review covers all core marketplace functionalities, including NFT minting, marketplace listing, NFT purchasing, and listing cancellation. Each operation is analysed to determine whether ownership integrity, resource safety, and transaction correctness are preserved throughout the NFT lifecycle.

7.2 Aptos Security Architecture

Aptos is built upon the Move programming language, which introduces **resource-oriented programming** as the foundation for secure digital asset management. Unlike conventional programming languages where objects can often be copied or implicitly deleted, Move resources enforce strict ownership semantics. Every resource must have exactly one owner, cannot be duplicated, and cannot be discarded unless explicitly destroyed according to the program logic.

The marketplace implementation consists of several logical components responsible for asset creation, marketplace management, and transaction execution.

The major components include:

Component	Purpose
NFT Module	Creates and manages NFT resources
Marketplace Module	Stores marketplace listings and executes trades
Listing Resource	Represents active marketplace listings

Component	Purpose
User Account Resources	Maintain ownership of NFTs
Payment Resources	Manage transaction settlement

This architecture ensures that ownership information remains directly associated with blockchain accounts while allowing secure transfer of resources between participants through authenticated transactions.

7.3 NFT Module Security Review

The NFT module is responsible for creating new NFT resources and assigning ownership to the transaction sender.

Security Objectives

The NFT module must ensure that:

- Every NFT is uniquely identifiable.
- NFT resources cannot be duplicated.
- Ownership is assigned correctly.
- Metadata remains associated with the NFT.
- Unauthorized users cannot modify another user's NFT.

Audit Findings

The audit confirmed that NFT creation follows Move's resource semantics.

Each successful mint operation results in:

- Creation of a new NFT resource.
- Assignment of ownership to the creator.
- Storage of metadata within the resource.
- Registration of ownership under the creator's account.

Because Move resources cannot be copied or implicitly duplicated, accidental duplication of NFTs is prevented by the language itself.

The audit found no evidence of unauthorized resource creation or ownership inconsistencies during functional testing.

Security Assessment

Property	Result
Resource Uniqueness	Passed
Ownership Assignment	Passed
Metadata Integrity	Passed
Duplicate Prevention	Passed
Unauthorized Modification	Passed

Risk Level: Very Low

7.4 Marketplace Module Security Review

The Marketplace module coordinates the listing, purchase, and removal of NFTs from the marketplace.

Since it manages ownership transitions and financial transactions, it represents the most security-sensitive component of the Aptos implementation.

The audit examined each public entry function individually.

7.4.1 NFT Listing

The listing function enables NFT owners to place their assets on the marketplace.

The following validations were confirmed:

- Transaction signer owns the NFT.
- NFT exists.
- Listing price is valid.
- Marketplace resource updates correctly.
- Duplicate listings are prevented.

Marketplace resources remained internally consistent throughout all tested transactions.

Unauthorized listing attempts were rejected because ownership verification is performed before marketplace updates occur.

Audit Result

Security Property

Ownership Validation

Signer Verification

Duplicate Listing Prevention

Marketplace Consistency

7.4.2 NFT Purchase

The purchase operation transfers ownership from the seller to the buyer while updating marketplace resources.

The audit confirmed that the function performs the following sequence:

1. Verify buyer transaction signature.
2. Validate listing existence.
3. Transfer payment.
4. Transfer NFT ownership.
5. Remove marketplace listing.
6. Update marketplace resources.

Move transaction execution guarantees atomicity, ensuring that either the complete purchase succeeds or all intermediate changes are rolled back.

No inconsistent ownership states were observed.

Security Assessment

Property

Payment Validation

Ownership Transfer

Listing Removal

Atomic Execution

Property

Resource Consistency

7.4.3 Listing Cancellation

Listing cancellation allows sellers to remove active marketplace entries.

The audit verified that:

- Only the seller may cancel.
- Listing resources are removed correctly.
- NFT ownership remains unchanged.
- Invalid cancellation requests are rejected.

No unauthorized listing removal was observed.

7.5 Resource Safety Analysis

Resource safety represents one of the strongest security guarantees provided by the Move language.

The audit confirmed that marketplace resources satisfy the following properties:

- Resources cannot be copied.
- Resources cannot be accidentally deleted.
- Resources always have a unique owner.
- Ownership transfers occur explicitly.
- Resources remain valid throughout execution.

These guarantees eliminate numerous vulnerabilities commonly encountered in traditional smart contract development.

Examples include:

- Duplicate NFTs
- Asset loss
- Ownership inconsistency
- Accidental resource destruction

Move's linear type system enforces these guarantees during compilation before contracts are deployed.

7.6 Signer Authentication Review

Unlike conventional applications where authentication may depend upon passwords or sessions, Aptos authenticates transactions using cryptographic signatures.

Every marketplace operation requiring authorization receives a **signer** reference representing the authenticated transaction sender.

The audit verified that signer authentication is enforced for all privileged operations.

Operation	Signer Required
-----------	-----------------

Mint NFT	Yes
----------	-----

List NFT	Yes
----------	-----

Buy NFT	Yes
---------	-----

Cancel Listing	Yes
----------------	-----

The use of signer authentication prevents unauthorized users from invoking marketplace functions on behalf of other participants.

7.7 Access Control Review

The marketplace implements role-based authorization through ownership verification rather than predefined administrative roles.

The audit confirmed the following authorization policies:

Marketplace Operation	Authorized User
-----------------------	-----------------

Mint NFT	Creator
----------	---------

List NFT	NFT Owner
----------	-----------

Purchase NFT	Buyer
--------------	-------

Cancel Listing	Seller
----------------	--------

No privilege escalation vulnerabilities were identified during manual review or functional testing.

7.8 Transaction Integrity

The integrity of marketplace transactions was evaluated throughout the complete NFT lifecycle.

The audit verified that every successful transaction preserves:

- NFT uniqueness.
- Ownership correctness.
- Marketplace consistency.
- Payment correctness.
- Resource validity.

Failed transactions automatically revert intermediate state changes, preventing inconsistent marketplace records.

This behavior significantly improves marketplace reliability.

7.9 Resource Account Protection

The Aptos implementation relies on Move resources associated with blockchain accounts.

The audit evaluated the security of account-resource relationships.

The following properties were verified:

- Resources remain associated with their owner accounts.
- Unauthorized accounts cannot access protected resources.
- Resource transfers occur only through validated marketplace functions.
- Invalid resource references are rejected.

Move's ownership semantics substantially reduce risks associated with unauthorized account manipulation.

7.10 Testnet Security Validation

Following successful compilation and deployment, the marketplace was executed on the Aptos Testnet.

Functional validation confirmed successful execution of:

- NFT Minting
- NFT Listing
- NFT Purchase
- Listing Cancellation

All marketplace operations completed successfully without unexpected ownership transitions or resource inconsistencies.

Benchmark execution also demonstrated:

- **100% transaction success rate (20/20 successful transactions).**
- Stable execution across multiple benchmark iterations.
- Consistent transaction validation.
- Reliable marketplace state updates following each operation.

These observations provide strong evidence that the implementation behaves correctly under realistic blockchain conditions.

7.11 Security Observations

Several positive security characteristics were identified during the review.

Strengths

- Strong resource ownership guarantees.
- Compile-time resource verification.
- Signer-based authentication.
- Atomic transaction execution.
- Prevention of duplicate NFTs.
- Consistent marketplace state.
- Reliable ownership transfers.
- Successful public testnet deployment.
- Stable benchmark execution with 100% success rate.

Minor Observations

No critical or high-severity vulnerabilities were identified.

The following improvements could further strengthen the implementation:

- Increase unit test coverage for invalid transaction scenarios.
- Improve event logging for marketplace analytics.
- Introduce additional validation for boundary values such as zero-price listings.
- Expand formal Move specifications to verify marketplace invariants.
- Implement fuzz testing to evaluate unexpected input combinations.

These observations are recommendations for improving assurance and maintainability rather than addressing functional security issues.

7.12 Comparison with the Sui Implementation

Although both Sui and Aptos are built using the Move language, their security architectures differ in several respects.

Sui relies on an object-centric model in which each NFT is represented as an independently owned object, allowing object ownership and transfer to be managed directly by the runtime. This design naturally supports fine-grained ownership tracking and enables parallel execution of transactions involving unrelated objects.

In contrast, Aptos adopts an account-centric resource model where NFTs exist as resources stored under blockchain accounts. Ownership is enforced through resource semantics and signer authentication rather than explicit object ownership. While this approach does not provide the same object-level parallelism as Sui, it offers similarly strong guarantees regarding uniqueness, resource safety, and transaction correctness.

From a security perspective, both implementations benefit from Move's linear type system, which prevents resource duplication and accidental destruction. Consequently, both platforms effectively mitigate common vulnerabilities such as duplicate asset creation, unauthorized ownership changes, and inconsistent resource management. The primary distinction lies in how ownership and state are represented and managed rather than in the strength of their security guarantees.

7.13 Aptos Security Summary

The security audit demonstrates that the Aptos implementation provides a secure and reliable foundation for the NFT marketplace. The combination of Move's resource-oriented programming model, signer-based authentication, and atomic transaction execution ensures

that NFT ownership remains protected throughout the complete marketplace lifecycle. The implementation successfully enforces access control, preserves resource integrity, and maintains consistent marketplace state during minting, listing, purchasing, and cancellation operations.

No critical or high-severity vulnerabilities were identified during source code review, functional testing, or public testnet validation. The implementation consistently prevented unauthorized operations, rejected invalid transactions, and maintained ownership correctness across all evaluated scenarios. Benchmark execution further confirmed the stability of the marketplace, achieving a 100% transaction success rate under the tested workload.

Overall, the Aptos implementation exhibits a **high level of application security** and demonstrates that the Move language's resource-oriented model provides strong safeguards against common smart contract vulnerabilities. While additional formal verification, fuzz testing, and expanded negative test coverage would further increase confidence for production deployment, the current implementation is well suited for academic research, comparative benchmarking, and public testnet use.

Chapter 8: Solana Program Security Review

8.1 Introduction

The Solana implementation of the NFT marketplace was developed using the **Rust programming language** and the **Anchor framework**, and deployed on the **Solana Devnet/Testnet**. Unlike Sui and Aptos, which employ Move's resource-oriented programming model, Solana follows an **account-based execution model** where all application state is stored within blockchain accounts owned by deployed programs. This architectural difference requires developers to explicitly validate account ownership, signer authorization, and account relationships during every instruction execution.

The objective of this chapter is to evaluate the security of the Solana implementation by examining its account management, Program Derived Address (PDA) usage, instruction validation, access control mechanisms, transaction integrity, and deployment behavior. The audit determines whether the implemented marketplace correctly enforces ownership, protects marketplace state, and prevents unauthorized manipulation of NFT assets throughout their lifecycle.

The review covers the complete marketplace workflow, including NFT minting, listing, purchasing, and delisting, while also evaluating platform-specific security controls provided by the Anchor framework.

8.2 Solana Security Architecture

Solana adopts a high-performance account-based architecture where executable programs are separated from application data. Programs themselves are stateless, while all persistent information—including NFT metadata, marketplace listings, and user information—is stored within blockchain accounts.

The marketplace implementation consists of the following major account types:

Account Type	Purpose
NFT Metadata Account	Stores NFT metadata
Marketplace Listing Account	Stores active listing information
Seller Account	Maintains seller ownership
Buyer Account	Maintains buyer information
Program Derived Address (PDA)	Stores marketplace-controlled state

Account Type	Purpose
System Program	Account creation and rent management
Token Program	Token transfers and ownership

Unlike Move-based blockchains where ownership is enforced by the language runtime, Solana requires every instruction to explicitly verify that the provided accounts satisfy the expected ownership and permission requirements.

The Anchor framework simplifies this process by allowing developers to define account constraints that are automatically verified before instruction execution.

8.3 NFT Module Security Review

The NFT module is responsible for creating NFT metadata and associating it with the creator.

Security Objectives

The module must ensure:

- Every NFT has a unique identity.
- Metadata remains consistent.
- Ownership is assigned correctly.
- Duplicate NFT creation is prevented.
- Unauthorized modification is impossible.

Audit Findings

The audit confirmed that NFT creation successfully initializes metadata accounts while assigning ownership to the creator.

The following security properties were verified:

- NFT metadata accounts are initialized correctly.
- Ownership information is stored accurately.
- Metadata cannot be modified without authorization.
- Duplicate account initialization is prevented.

Because every NFT is represented through a unique account structure, accidental duplication was not observed during testing.

Security Assessment

Property	Result
NFT Initialization	Passed
Ownership Assignment	Passed
Metadata Integrity	Passed
Duplicate Prevention	Passed

Risk Level: Low

8.4 Marketplace Program Security Review

The marketplace program manages NFT listings, purchases, and delisting operations.

Since these operations directly modify marketplace state, they represent the primary focus of the security audit.

8.4.1 NFT Listing

Listing an NFT creates or updates marketplace information stored in PDA-controlled accounts.

The audit verified that listing operations perform the following checks:

- Seller account verification.
- NFT ownership verification.
- PDA validation.
- Listing account initialization.
- Marketplace state update.

Unauthorized listing attempts using invalid accounts were rejected through account ownership verification.

Security Assessment

Property

Account Ownership Validation

PDA Verification

Seller Authorization

Marketplace Update

8.4.2 NFT Purchase

The purchase operation transfers ownership from the seller to the buyer while updating marketplace state.

The audit confirmed that successful purchase execution performs:

1. Buyer verification.
2. Seller verification.
3. Listing validation.
4. Payment transfer.
5. Ownership update.
6. Listing removal.

The transaction executes atomically, ensuring that incomplete ownership transfers cannot occur.

Security Properties

Property

Buyer Authentication

Seller Verification

Ownership Transfer

Listing Removal

Atomic Transaction

8.4.3 NFT Delisting

The delisting instruction allows sellers to remove NFTs from the marketplace.

The audit confirmed:

- Only sellers may delist NFTs.
- Listing accounts are closed correctly.
- Marketplace state remains consistent.
- NFT ownership remains unchanged.

No unauthorized delisting behavior was observed.

8.5 Program Derived Address (PDA) Security Review

Program Derived Addresses (PDAs) represent one of the most important security mechanisms in Solana applications.

PDAs are deterministic accounts generated from predefined seeds and controlled exclusively by the deployed program.

The audit evaluated:

- PDA derivation correctness.
- Seed consistency.
- Ownership validation.
- Unauthorized PDA access.

The following observations were made:

- All marketplace PDAs were generated deterministically.
- PDA ownership remained under program control.
- External users could not arbitrarily modify PDA contents.
- Marketplace state remained protected.

Proper PDA usage significantly reduces the risk of account spoofing and unauthorized marketplace modification.

8.6 Account Ownership Verification

Unlike Move, Solana requires explicit verification of account ownership.

The audit reviewed every instruction to confirm that:

- NFT accounts belong to expected owners.
- Marketplace accounts belong to the deployed program.
- Mutable accounts are correctly declared.
- Read-only accounts cannot modify state.

Failure to perform these checks would allow unauthorized account manipulation.

The implementation correctly validates account ownership before executing marketplace logic.

8.7 Signer Authentication Review

Every privileged Solana instruction requires one or more authenticated signers.

The audit verified signer validation for:

Operation Signer Required

Mint NFT Creator

List NFT Seller

Buy NFT Buyer

Delist NFT Seller

Anchor automatically validates signer constraints before instruction execution.

No missing signer validations were identified.

8.8 Anchor Constraint Validation

The Anchor framework provides declarative account constraints that simplify secure program development.

The audit confirmed correct usage of Anchor constraints including:

- Signer
- Account

- Program
- SystemAccount
- Mutable account constraints
- PDA seed verification

These constraints eliminate many common implementation errors by performing validation before business logic executes.

8.9 Transaction Integrity

Marketplace operations involve multiple accounts participating within a single transaction.

The audit verified that transactions preserve:

- NFT ownership.
- Marketplace consistency.
- Account validity.
- Payment correctness.
- Listing integrity.

Solana's transaction model guarantees atomic execution, ensuring that partially completed marketplace operations cannot corrupt application state.

8.10 Testnet Security Validation

Following successful program compilation, the marketplace was deployed to the Solana Devnet/Testnet.

Functional testing confirmed successful execution of:

- NFT Minting
- NFT Listing
- NFT Delisting

(The buy operation was excluded from benchmarking because it required a second funded wallet; however, functional testing confirmed that the purchase logic operated correctly under the intended workflow.)

Benchmark execution demonstrated:

- Successful deployment.
- Stable instruction execution.
- Consistent account validation.
- Correct PDA behavior.
- Reliable ownership management.

No unexpected account inconsistencies or transaction failures were observed during testing.

8.11 Security Observations

The audit identified several strengths of the Solana implementation.

Strengths

- Strong PDA-based state management.
- Reliable signer verification.
- Proper account ownership validation.
- Secure transaction atomicity.
- Anchor framework constraint enforcement.
- Successful deployment and execution on testnet.
- Deterministic account initialization.

Minor Observations

No critical or high-severity vulnerabilities were identified.

Several improvements could further strengthen the implementation:

- Expand validation of optional account parameters.
- Improve custom error messages for failed account validation.
- Increase automated testing for malformed account inputs.
- Perform fuzz testing on PDA seed generation.
- Add additional event logging for marketplace analytics.

These observations improve maintainability and auditability rather than addressing security flaws.

8.12 Comparison with Move-Based Implementations

The Solana implementation differs fundamentally from the Sui and Aptos implementations due to its account-based execution model.

In Sui and Aptos, Move's resource-oriented programming language automatically enforces ownership and resource safety, preventing duplication or accidental destruction of digital assets at compile time. Solana, however, delegates much of this responsibility to the developer. Correctness depends on explicit validation of accounts, signers, ownership, and Program Derived Addresses.

Despite this difference, the Anchor framework substantially reduces implementation complexity by automatically enforcing account constraints before program logic executes. When used correctly, these constraints provide strong protection against unauthorized account access and privilege escalation.

From a security perspective, the Solana implementation achieved functionality equivalent to the Move-based implementations while relying on different architectural mechanisms. The audit found that the implemented account validation, PDA management, and signer authentication successfully preserved ownership integrity and marketplace consistency throughout the evaluated transaction flows.

8.13 Solana Security Summary

The security review demonstrates that the Solana implementation provides a secure and reliable foundation for the NFT marketplace when developed using the Anchor framework and established Solana programming practices. The implementation correctly enforces signer authentication, validates account ownership, protects marketplace state through Program Derived Addresses, and preserves transaction integrity across NFT minting, listing, purchasing, and delisting operations.

Manual code review, functional testing, and public testnet deployment did not reveal any critical or high-severity vulnerabilities. The combination of Anchor's declarative account constraints and explicit ownership verification effectively mitigates common risks associated with account-based blockchain programming, including unauthorized state modification, account spoofing, and privilege escalation.

While Solana requires more explicit security checks than Move-based platforms, the implemented controls successfully maintain marketplace correctness and ownership integrity. Future improvements, including expanded negative testing, fuzz testing, enhanced event logging, and independent third-party audits, would further strengthen production readiness. Overall, the Solana implementation demonstrates a **high level of application security** and provides a solid basis for comparative evaluation alongside the Sui and Aptos implementations.

Chapter 9: Cross-Chain Security Considerations

9.1 Introduction

The implementation of the NFT marketplace across **Sui**, **Aptos**, and **Solana** provides an opportunity to evaluate how different blockchain architectures influence application security. Although the three implementations deliver the same functional capabilities—including NFT minting, listing, purchasing, and delisting—they employ distinct execution models, ownership mechanisms, and validation strategies. These architectural differences directly affect how security is enforced and how common blockchain vulnerabilities are mitigated.

The purpose of this chapter is to compare the security characteristics of the three blockchain platforms from an application development perspective. Rather than identifying which platform is inherently more secure, the comparison focuses on how each platform supports secure smart contract development, protects digital assets, and assists developers in preventing implementation errors.

9.2 Comparison of Security Models

The most significant difference among the three platforms lies in how ownership and state are represented.

Sui adopts an **object-centric execution model**, where every NFT is treated as an independent on-chain object with a unique identifier and a single owner. Ownership is explicitly maintained by the blockchain, allowing transactions involving unrelated objects to execute independently. This approach naturally enforces ownership integrity and simplifies asset management while supporting parallel transaction execution.

Aptos employs a **resource-oriented account model** built on the Move language. NFTs are represented as resources stored within user accounts. Move's linear type system guarantees that resources cannot be copied, implicitly deleted, or duplicated, thereby preserving ownership consistency throughout the asset lifecycle. Authentication is primarily achieved through signer verification.

Solana follows an **account-based architecture**, where all marketplace data resides within blockchain accounts controlled by deployed programs. Security depends on explicit verification of account ownership, signer authorization, and Program Derived Addresses (PDAs). While this model offers greater flexibility, it places more responsibility on developers to correctly implement security checks.

Table 9.1 summarizes the primary architectural differences.

Table 9.1 Security Architecture Comparison

Security Aspect	Sui	Aptos	Solana
Programming Language	Move	Move	Rust (Anchor)
Execution Model	Object-Oriented	Resource-Oriented	Account-Based
Ownership Model	Object Ownership	Resource Ownership	Account Ownership
Authentication	Capabilities & Signer	Signer	Signer + Account Validation
State Storage	Objects	Resources	Accounts
Parallel Execution	Object-Level	Transaction-Level	Runtime Scheduler
Compile-Time Resource Safety	Yes	Yes	No (Developer Enforced)

The comparison illustrates that Sui and Aptos derive many security guarantees directly from the Move language, whereas Solana relies more heavily on framework-assisted runtime validation.

9.3 Ownership Protection

Ownership protection is the most critical security requirement for an NFT marketplace, as unauthorized ownership changes directly compromise the integrity of digital assets.

Both Sui and Aptos leverage Move's resource-oriented programming model to ensure that ownership remains unique and cannot be duplicated. In Sui, NFTs exist as independently owned objects whose ownership is tracked by the blockchain runtime. In Aptos, ownership is enforced through resources associated with user accounts, with the language preventing unintended duplication or destruction.

In contrast, Solana stores ownership information within accounts and requires explicit verification during instruction execution. The implementation achieves this by validating account ownership, signer permissions, and PDA constraints before modifying marketplace state. Although more developer effort is required, the audit confirmed that these checks were correctly implemented.

Across all three implementations, no unauthorized ownership transfers or duplicate asset creation were observed during testing.

9.4 Access Control Mechanisms

Each blockchain employs a different approach to enforcing access control while achieving equivalent security outcomes.

Sui combines signer authentication with object ownership verification, ensuring that only the owner of an NFT can perform privileged operations such as listing or cancellation. Aptos relies on signer authentication together with resource ownership, allowing only the account controlling a resource to modify it. Solana enforces access control by validating transaction signers and ensuring that only authorized accounts satisfying predefined Anchor constraints can execute marketplace instructions.

Despite the implementation differences, the audit confirmed that all three platforms successfully enforce the following authorization policies:

- Only creators may mint new NFTs.
- Only owners may list NFTs.
- Only buyers may complete purchase transactions.
- Only sellers may cancel active listings.

No privilege escalation vulnerabilities were identified during the audit.

9.5 Common Security Strengths

The audit identified several security properties consistently implemented across all three blockchain platforms:

- Successful verification of transaction signatures.
- Protection against unauthorized ownership transfers.
- Atomic execution of marketplace transactions.
- Consistent marketplace state after transaction completion.
- Prevention of duplicate NFT ownership.
- Successful validation of marketplace operations on public testnets.

These shared characteristics demonstrate that, despite architectural differences, each implementation satisfies the core security requirements of a decentralized NFT marketplace.

9.6 Platform-Specific Security Strengths

While all three implementations achieved the project's security objectives, each platform offers unique advantages.

Sui benefits from its object-centric architecture, which naturally models NFTs as independently owned assets. The distinction between owned and shared objects simplifies ownership management while enabling efficient parallel execution of independent transactions.

Aptos provides strong compile-time guarantees through Move's resource-oriented programming model. Resource semantics eliminate several classes of ownership-related vulnerabilities and simplify reasoning about digital asset integrity.

Solana offers a flexible account-based architecture supported by the Anchor framework. Although developers must explicitly validate accounts and permissions, Anchor's declarative constraints significantly reduce the likelihood of common implementation mistakes. Program Derived Addresses further strengthen application security by ensuring that critical marketplace state remains under program control.

9.7 Platform-Specific Challenges

The audit also identified challenges specific to each platform.

In Sui, operations involving shared marketplace objects require consensus coordination, resulting in higher latency for listing and cancellation transactions. This is an architectural trade-off rather than a security weakness.

Aptos requires careful management of resources associated with user accounts. While the Move language provides strong guarantees, developers must understand resource semantics to avoid implementation errors during complex contract development.

Solana places greater responsibility on developers to implement comprehensive account validation. Missing signer checks, incorrect PDA derivation, or insufficient account ownership verification could introduce vulnerabilities if not carefully implemented. The Anchor framework mitigates many of these risks, but correctness ultimately depends on proper program design.

9.8 Cross-Chain Security Assessment

The comparative analysis indicates that all three implementations successfully preserve the essential security properties required for a decentralized NFT marketplace. No critical or

high-severity vulnerabilities were identified in any implementation during manual review, functional testing, or public testnet deployment.

Although the mechanisms differ, each platform effectively protects digital asset ownership, enforces access control, and maintains consistent marketplace state. Sui and Aptos derive much of their security from the Move language itself, while Solana achieves equivalent protection through explicit account validation and Anchor framework constraints.

From an application security perspective, the differences among the three implementations primarily concern developer experience, execution architecture, and validation mechanisms rather than the ability to implement a secure NFT marketplace.

9.9 Chapter Summary

The cross-chain comparison demonstrates that Sui, Aptos, and Solana each provide a secure foundation for decentralized NFT marketplace development when their recommended programming practices are followed. Sui excels in object ownership management and parallel execution, Aptos offers strong compile-time resource safety through the Move language, and Solana combines flexible account-based programming with robust framework-assisted validation.

While each platform introduces unique architectural considerations, the completed implementations consistently achieved the project's security objectives by protecting NFT ownership, enforcing authorization, maintaining marketplace integrity, and successfully validating all core operations on public testnets. These findings establish the basis for the consolidated security testing results and vulnerability assessment presented in the following chapters.

Chapter 10: Security Testing Results

10.1 Introduction

Following the security review of the Sui, Aptos, and Solana implementations, a series of security tests were conducted to validate the correctness and reliability of the developed NFT marketplace under practical operating conditions. The objective of these tests was to verify that the implemented security controls effectively enforce ownership, authorization, and state consistency while preventing unauthorized operations across the complete NFT lifecycle.

The testing process combined compilation verification, unit testing, functional validation, public testnet deployment, and execution of marketplace transactions. Unlike the source code review presented in the previous chapters, this chapter focuses on the observed outcomes of these tests and evaluates whether the implementations behaved as expected during execution.

10.2 Testing Environment

Security validation was performed independently on the three blockchain platforms using their respective development toolchains and public testnet environments. The tests were executed after successful compilation of the smart contracts and completion of implementation-specific unit tests.

The primary testing activities included:

- Smart contract compilation verification.
- Unit test execution.
- Public testnet deployment.
- Functional execution of marketplace operations.
- Authorization validation.
- Ownership verification.
- Transaction consistency checks.
- Negative testing using invalid transaction scenarios.

Conducting the evaluation on public testnets ensured that the contracts were executed under realistic blockchain conditions involving distributed validators, network latency, and persistent on-chain state.

10.3 Functional Validation Results

The primary objective of functional testing was to verify that each implementation correctly executes the complete NFT marketplace workflow while preserving security properties throughout transaction execution.

The following marketplace operations were successfully validated on all three blockchain implementations:

- NFT Minting
- NFT Listing
- NFT Purchase*
- NFT Delisting / Cancellation

Note: In the Solana benchmark, the purchase operation was not included in the latency measurements because it required a second funded wallet. However, the purchase functionality was verified separately during functional testing and behaved correctly.

Table 10.1 summarizes the functional validation results.

Table 10.1 Functional Security Validation

Operation	Sui	Aptos	Solana
NFT Minting	Success	Success	Success
NFT Listing	Success	Success	Success
NFT Purchase	Success	Success	Success
Listing Cancellation / Delisting	Success	Success	Success
Ownership Transfer	Success	Success	Success
Marketplace State Update	Success	Success	Success

The successful execution of these operations confirms that the developed marketplace preserves ownership integrity and correctly manages marketplace state across all three blockchain platforms.

10.4 Compilation and Unit Testing

Compilation represents the first stage of security validation by ensuring that the smart contracts satisfy the language-specific safety rules before deployment.

For the **Sui** implementation, both package compilation and Move unit tests completed successfully.

- sui move build — Passed
- sui move test — Passed

Similarly, the **Aptos** implementation successfully compiled without resource or type errors, and all implemented Move unit tests executed successfully.

The **Solana** implementation compiled successfully using the Anchor framework, validating account definitions, instruction constraints, and program structure before deployment.

Successful compilation across all platforms provides confidence that the implementations satisfy their respective language-level safety requirements.

10.5 Authorization Testing

Authorization testing evaluated whether privileged marketplace operations could only be executed by legitimate users.

The following scenarios were verified:

Security Check	Result
Only creator can mint NFT	Success Passed
Only owner can list NFT	Success Passed
Only buyer can complete purchase	Passed
Only seller can cancel listing	Success Passed
Unauthorized operations rejected	Success Passed

No privilege escalation or unauthorized transaction execution was observed during testing.

10.6 Ownership Validation

Ownership validation represents one of the most critical aspects of NFT marketplace security.

Throughout testing, ownership was verified before and after every marketplace operation.

The following properties were confirmed:

- Newly minted NFTs were assigned to the creator.

- Listed NFTs remained associated with their owners until purchase.
- Ownership transferred correctly after successful purchase.
- Cancelled listings returned to normal ownership status.
- Duplicate ownership was never observed.

Across all tested scenarios, ownership records remained consistent with the expected marketplace workflow.

10.7 Marketplace State Consistency

Marketplace operations modify shared application state through the creation, update, and removal of marketplace listings.

The audit verified that marketplace state remained internally consistent following every transaction.

Specifically:

- Newly listed NFTs appeared in the marketplace.
- Purchased NFTs were automatically removed from active listings.
- Cancelled listings disappeared from the marketplace.
- Marketplace metadata remained synchronized with NFT ownership.

No orphaned listings, duplicate marketplace entries, or inconsistent marketplace records were observed.

10.8 Negative Testing

In addition to successful transaction execution, several invalid transaction scenarios were evaluated to determine whether the marketplace correctly rejected unauthorized or malformed requests.

Representative negative test cases included:

- Attempting to list an NFT without ownership.
- Attempting to cancel another user's listing.
- Attempting to purchase an unavailable NFT.
- Providing invalid transaction parameters.
- Referencing invalid object or account identifiers.

In each case, the corresponding blockchain implementation rejected the transaction without modifying marketplace state. These results indicate that the implemented validation logic effectively prevents unauthorized operations and protects the integrity of the marketplace.

10.9 Testnet Deployment Validation

Successful deployment on public testnets provides additional assurance that the smart contracts function correctly in a decentralized execution environment.

Deployment validation confirmed:

- Successful package deployment.
- Correct contract initialization.
- Successful execution of marketplace operations.
- Stable interaction with blockchain validators.
- Persistent storage of marketplace state.
- Reliable transaction confirmation.

No deployment-related security issues or execution failures were encountered during the evaluation.

10.10 Overall Testing Summary

The consolidated results of the security testing demonstrate that the developed NFT marketplace satisfies its intended security requirements across Sui, Aptos, and Solana. All core marketplace operations executed successfully during functional testing, ownership remained protected throughout the NFT lifecycle, and unauthorized transactions were consistently rejected.

The successful completion of compilation, unit testing, public testnet deployment, and end-to-end functional validation provides strong evidence that the implemented security mechanisms operate correctly in realistic blockchain environments. While further assurance could be obtained through formal verification, fuzz testing, and independent third-party audits, the current evaluation indicates that the marketplace implementation is secure, functionally correct, and suitable for academic research and comparative performance benchmarking.

Chapter 11: Vulnerability Assessment

11.1 Introduction

A vulnerability assessment is a systematic process of identifying, evaluating, and classifying weaknesses that may compromise the confidentiality, integrity, or availability of a system. Within blockchain applications, vulnerabilities often arise from incorrect implementation of ownership verification, insufficient authorization checks, improper state management, insecure account handling, or logical errors in transaction execution.

The objective of this assessment is to determine whether the developed NFT marketplace contains security weaknesses that could be exploited by malicious users during normal marketplace operations. The assessment is based on the manual code review, functional testing, testnet deployment, and platform-specific security analysis presented in the previous chapters.

Unlike commercial security audits that often report numerous informational findings, this assessment focuses on vulnerabilities that are relevant to the implemented marketplace and evaluates their practical impact within the scope of this project.

11.2 Vulnerability Classification Methodology

The identified observations were classified according to their potential impact on the security and functionality of the marketplace. A four-level severity model was adopted to provide a consistent basis for evaluating identified issues.

Table 11.1 Vulnerability Severity Levels

Severity	Description
Critical	A vulnerability that may lead to complete compromise of NFT ownership, unauthorized asset transfers, or permanent corruption of marketplace state.
High	A vulnerability that significantly affects marketplace security or allows unauthorized execution of privileged operations.
Medium	A weakness that does not directly compromise ownership but may reduce reliability, resilience, or maintainability under specific conditions.
Low	Minor issues that do not affect security but could improve robustness, usability, or code quality.
Informational	Recommendations and best practices intended to improve maintainability or future development without affecting current functionality.

This classification aligns with common practices adopted in smart contract security assessments.

11.3 Assessment Summary

The comprehensive audit of the Sui, Aptos, and Solana implementations did **not identify any Critical or High severity vulnerabilities**.

The security mechanisms implemented across all three platforms successfully enforce:

- Ownership verification
- Access control
- Transaction authorization
- Marketplace state consistency
- Secure NFT transfers

The observations identified during the assessment relate primarily to opportunities for improving testing coverage, monitoring, event logging, and future maintainability rather than weaknesses that could compromise the security of the deployed marketplace.

The overall vulnerability distribution is summarized in Table 11.2.

Table 11.2 Vulnerability Distribution

Severity	Number of Findings
Critical	0
High	0
Medium	2
Low	4
Informational	5

11.4 Medium Severity Observations

Although no exploitable vulnerabilities were identified, two medium-severity observations were noted that could affect the robustness of future deployments if left unaddressed.

Finding M-01: Limited Negative Test Coverage

Severity: Medium

Description

The implemented unit and functional tests primarily validate successful transaction execution. While representative invalid transaction scenarios were evaluated, the automated test suite could be expanded to include a broader range of negative and boundary conditions.

Examples include:

- Extremely large input values.
- Invalid marketplace object references.
- Repeated cancellation attempts.
- Simultaneous listing requests.
- Stress testing under high transaction volumes.

Potential Impact

Limited negative testing increases the possibility that edge-case behaviors remain undiscovered during development.

Recommendation

Expand automated unit tests to include:

- Boundary value analysis.
- Invalid ownership scenarios.
- Concurrent transaction simulations.
- Property-based testing.
- Regression testing for future updates.

Finding M-02: Limited Formal Verification Coverage

Severity: Medium

Description

Although the Move implementations benefit from compile-time resource safety and successful unit testing, the marketplace logic was not exhaustively specified using formal verification assertions for all business invariants.

Potential Impact

Complex logical errors that are difficult to detect through conventional testing may remain undiscovered.

Recommendation

Future versions should incorporate more extensive Move Prover specifications (where applicable) to formally verify critical properties such as:

- Ownership invariants.
- Marketplace consistency.
- Listing uniqueness.
- Transaction correctness.

11.5 Low Severity Observations

Several low-severity observations were identified during the audit.

These findings do not represent security vulnerabilities but may improve maintainability, monitoring, and operational visibility.

Finding L-01: Event Logging

Marketplace operations currently generate only the events required by the implementation.

Additional events for marketplace analytics could improve monitoring and auditing.

Recommendation

Emit events for:

- Listing creation.
- Purchase completion.
- Listing cancellation.
- Loyalty reward updates.

Finding L-02: Error Reporting

Some transaction failures rely primarily on framework-generated error messages.

More descriptive application-specific error codes would improve debugging and developer experience.

Finding L-03: Test Coverage Expansion

Current testing successfully validates the complete NFT lifecycle.

Additional automated regression tests would improve confidence during future feature development.

Finding L-04: Benchmark Diversity

Performance benchmarking focused on sequential CLI execution.

Future evaluations could include:

- Concurrent execution.
- Larger transaction batches.
- Long-duration stress testing.
- Multi-user workloads.

Although this observation primarily affects benchmarking rather than security, broader testing would improve confidence in operational behavior.

11.6 Informational Observations

The following observations are provided as recommendations for future enhancement rather than security issues.

- Introduce continuous integration for automated smart contract testing.
- Expand documentation of security assumptions.
- Increase code comments for complex marketplace logic.
- Maintain version-controlled deployment configurations.
- Perform periodic third-party security reviews before production deployment.

These recommendations are intended to support long-term maintainability and project evolution.

11.7 Security Controls Successfully Verified

During the assessment, several important security controls were successfully validated across all three blockchain implementations.

Table 11.3 Verified Security Controls

Security Control	Sui	Aptos	Solana	Status
Ownership Verification	Success	Success	Success	Passed
Access Control	Success	Success	Success	Passed
Transaction Authentication	Success	Success	Success	Passed
Marketplace State Validation	Success	Success	Success	Passed
Duplicate NFT Prevention	Success	Success	Success	Passed
Secure Ownership Transfer	Success	Success	Success	Passed
Transaction Atomicity	Success	Success	Success	Passed
Testnet Validation	Success	Success	Success	Passed

The successful verification of these controls significantly reduces the likelihood of common NFT marketplace vulnerabilities and demonstrates the effectiveness of the implemented security architecture.

11.8 Overall Security Assessment

The completed assessment indicates that the NFT marketplace exhibits a **strong application-level security posture** across all three blockchain implementations. The combination of Move's resource safety in Sui and Aptos, together with explicit account validation and Anchor framework constraints in Solana, effectively protects NFT ownership, enforces authorization, and maintains marketplace integrity.

No vulnerabilities capable of compromising NFT ownership, enabling unauthorized transactions, or corrupting marketplace state were identified during manual code review, functional testing, or public testnet validation.

The remaining observations primarily concern opportunities for improving verification coverage, operational monitoring, and long-term maintainability rather than correcting implementation flaws.